



DPLoRA: Stable Diffusion Model distillation via Dynamic Parallel LoRA branches

Weibin Zeng^{a,1}, Yan Kong^{a,1}, Fuzhang Wu^{a,*}, Youheng Ren^a, Sicheng Shen^a, Yuqing Fan^a, Weiming Dong^b

^a Institute of Software, Chinese Academy of Sciences, Beijing, China

^b MAIS, Institute of Automation, Chinese Academy of Sciences, Beijing, China

ARTICLE INFO

Keywords:

Stable Diffusion Model
Model distillation
Low rank adaptation
Image generation

ABSTRACT

Industrial applications often employ multiple Stable Diffusion Models (SDMs) to meet diverse user demands. While this approach is effective, it inevitably incurs substantial memory costs and switching overhead. We observe that many SDMs are fine-tuned derivatives of a common base model, and that variations among them stem from parameter updates during domain adaptation—that is, each SDM acquires domain-specific knowledge through a domain-specific gradient subspace. While mapping these subspaces onto lightweight adapters like LoRA offers a promising solution, standard LoRA suffers from fixed-rank constraints and requires heuristic manual tuning, limiting its ability to capture domain knowledge. Therefore, we present **Dynamic Parallel LoRA (DPLoRA)**, a lightweight adaptation framework built upon LoRA principles. DPLoRA distills domain knowledge from fine-tuned SDMs by automatically identifying their domain gradients while minimizing inference overhead. Specifically, DPLoRA dynamically learns the exclusive domain space via parallel LoRA branches with adaptive importance factors and employs a reparameterization strategy to reduce inference memory without performance degradation. The distilled lightweight domain-specialized DPLoRA modules can be seamlessly plugged into a single general base model, enabling efficient handling of diverse inputs by swapping the corresponding DPLoRA modules. Extensive experiments across multiple datasets and widely-used fine-tuned models demonstrate the practicality and effectiveness of our proposed method.

1. Introduction

Diffusion models [1] have demonstrated remarkable capabilities in high-fidelity visual synthesis, driving rapid progress in applications such as text-to-image (Imagen [2], LDM [3]), text-to-video (Text2Video-Zero [4]), and text-to-3D generation (Magic3D [5]). Among various implementations, Stable Diffusion (SDM), built upon Latent Diffusion Models (LDM) [3], has emerged as the most widely used generation model due to its relative efficiency in the latent space. In many practical scenarios, multiple SDMs are deployed to address diverse user demands and facilitate customized generation (see Fig. 1(a)). However, the inherent complexity of SDM architectures still imposes heavy demands on computational resources, particularly in terms of GPU memory consumption, rendering their large-scale deployment increasingly challenging.

Substantial research has been devoted to enhancing diffusion models (DMs) while reducing computational demands, primarily through

quantization [6], pruning [7], and distillation [8]. Quantization decreases memory consumption by compressing parameter bit-widths, often at the expense of precision; pruning eliminates redundant parameters but may degrade performance due to structural modifications; and distillation trains a compact student model to mimic a larger teacher, thereby accelerating inference. **However, these methods focus on optimizing individual SDMs, and storing or deploying a multitude of separate models for diverse demands remains operationally expensive.** Notably, most community-driven SDMs (e.g., those on Civitai² and HuggingFace³) are fine-tuned from a shared foundation model, such as Stable-Diffusion-V1.5 (SD1.5) [3]. This implies that these models share substantial common knowledge, with their model variations stemming primarily from domain-specific knowledge acquired through the optimization of domain-specific gradient matrices during fine-tuning.

* Corresponding author.

E-mail address: fuzhang@iscas.ac.cn (F. Wu).

¹ These authors contributed equally to this work.

² <https://civitai.com/>

³ <https://huggingface.co/>

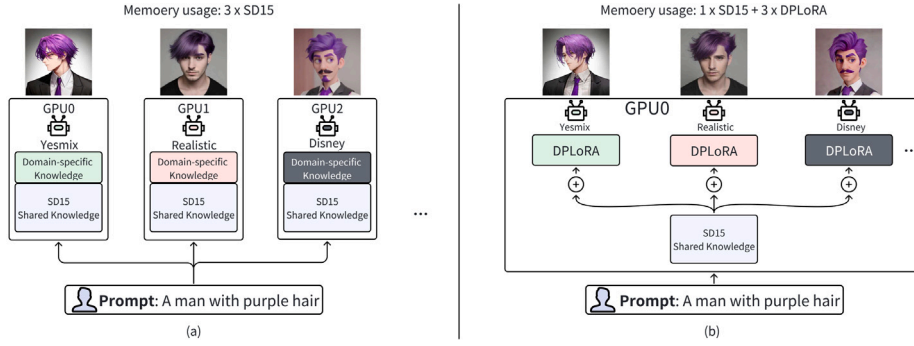


Fig. 1. Here we consider a scenario where users generate images in multiple styles using different SDMs. (a) Due to their complex architectures, SDMs must be deployed on separate GPUs to meet inference speed requirements. (b) By decomposing SDMs into a shared base model and lightweight DPLoRA modules, memory consumption can be significantly reduced.

Building on this insight, we propose a novel diffusion model distillation framework that decomposes fine-tuned SDMs into a shared foundational model and personalized, lightweight adapters. During training, only the adapters are updated to learn domain-specific knowledge. After training, the framework stores a single backbone along with multiple lightweight adapters, our approach allows efficient adaptation across diverse domains (see Fig. 1(b)). This framework reduces model storage requirements while, more importantly, minimizing memory consumption in multi-SDM deployment scenarios, thereby significantly mitigating the dependence on high-resource environments.

To realize this framework, we adopt SD1.5 [3] as the foundational model. For the adapter, low-rank adaptation (LoRA) [9] is an intuitive choice. It builds upon the hypothesis that parameter updates during fine-tuning, which are driven by the native gradient g , reside within a low-rank intrinsic space. By freezing the original weights $\theta_0 \in \mathbb{R}^{m \times n}$, LoRA represents the parameter updates as the product of two low-rank matrices: $\theta \approx \theta_0 + AB^T$, where $A \in \mathbb{R}^{m \times d}$ and $B \in \mathbb{R}^{n \times d}$. Crucially, optimizing A and B with their respective gradients g_A and g_B implies that the full parameter matrix θ is implicitly updated along a low-rank gradient direction, formulated as $g_{lora} = g_A B^T + A g_B^T$. This mechanism establishes LoRA as a parameter-efficient proxy for full fine-tuning, where its underlying objective is to effectively approximate the target gradient g via the induced low-rank gradient g_{lora} .

Based on this perspective, each fine-tuned SDM can be characterized by a domain-specific gradient subspace g_{sdm} , along which full fine-tuning of the base model can recover its specific performance. To achieve comparable performance, LoRA seeks to approximate g_{sdm} by adjusting its own gradient subspace g_{lora} through manual rank tuning, where the approximation fidelity determines the effectiveness of learned domain knowledge. However, this process introduces several limitations: (i) The alignment between g_{lora} and g_{sdm} is heuristic, relying on empirical tuning and lacking guarantees of optimal adaptation across tasks. (ii) The rank of the incremental matrix Δ is pre-specified, ignoring the heterogeneous importance of weights across layers and modules. (iii) Expanding the gradient subspace in LoRA may improve generation quality, but it also enlarges the adapter, leading to higher memory usage and slower inference, thereby introducing a performance-efficiency trade-off.

To address these limitations, we present an innovative method based on LoRA, termed **Dynamic Parallel LoRA (DPLoRA)**. DPLoRA introduces parallel low-rank branches with learnable importance factors, allowing each layer to dynamically adjust its update direction during training according to its task-specific contribution. This design avoids the challenges of applying a uniform configuration to incremental matrices Δ across all layers. Specifically, DPLoRA equips each module with multiple parallel low-rank update branches, where the contribution of each branch is controlled by a learnable importance factor. Such a mechanism guides the resulting gradient g_{dplora} to better align with the gradient subspace of the target SDM g_{sdm} , thereby

matching the actual requirements of each layer in downstream tasks. In contrast to conventional LoRA methods that rely on manually tuned hyperparameters, DPLoRA adaptively optimizes gradient directions, reducing heuristic dependency and significantly improving adaptation performance across downstream tasks. Additionally, by leveraging a re-parameterization technique, DPLoRA facilitates the exploration of a larger gradient space without extra inference cost, ensuring both effectiveness and efficiency.

Our main contributions are summarized as follows:

1. We propose a novel distillation framework that decomposes fine-tuned SDMs into a shared base model and domain-specific components. By reusing the base model, computational cost is significantly reduced in multi-SDM deployment.
2. We introduce DPLoRA, which leverages reparameterized parallel LoRA branches with learnable factors to automatically optimize the gradient adaptation trajectory over a wide range, without incurring additional storage or inference costs.
3. We theoretically analyze the equivalent gradient subspaces of LoRA, Naive Parallel LoRA, and DPLoRA, showing that DPLoRA achieves a higher rank upper bound under the same number of trainable parameters, leading to stronger representational capacity and domain knowledge acquisition.
4. We conduct comprehensive experiments across multiple datasets to validate the efficacy of the proposed method on both widely-used fine-tuned SDMs and Large Language Models.

2. Related works

Diffusion models. Diffusion models (DMs) [1] have emerged as a powerful paradigm for image generation, surpassing traditional approaches such as GAN [10] and StyleGAN [11]. By leveraging the capabilities of CLIP and efficient latent-space Variational AutoEncoders (VAE), SDMs [3] have achieved remarkable performance in high-resolution image generation. Specifically, SDMs perform the diffusion and denoising processes in the latent space, where a parameterized network $\epsilon_\theta(x_t, t)$ is trained to predict the noise added at timestep t under a denoising score matching objective. During inference, images are generated by iteratively reversing the diffusion process from Gaussian noise. With the increasing adoption of diffusion models, there is a growing demand for personalized content generation. While DreamBooth [12] is widely used for high-quality subject-driven synthesis, its reliance on full fine-tuning leads to excessive computational and storage overhead, necessitating more scalable adaptation and model compression techniques.

Model compression. Model compression is crucial for scaling large models, with knowledge distillation (KD) serving as a core technique. While distillation has matured for traditional vision tasks, it has recently been extended to diffusion models (DMs) for efficient generative modeling. Progressive distillation [8] obtains a new diffusion model that requires only half the number of sampling steps. BK-SDM [13] prunes redundant U-Net blocks via structural reduction. MobileDiffusion [14] optimizes the network architecture specifically for on-device generation to achieve near-instant text-to-image synthesis. Recent advancements in distillation have introduced even more sophisticated mechanisms to preserve performance. For instance, WaDi [15] explores weight direction-aware distillation for one-step image synthesis, while the LIFT and PLACE framework [16] adopts coarse-to-fine knowledge distillation for lightweight diffusion models. DLIENet [17] leverages bottleneck and attention-based distillation to enhance feature representation, while DTSKD [18] utilizes dual teachers to refine multi-branch outputs. Furthermore, CapDistill [19] adopts a hierarchical strategy to transfer object- and action-level semantics. However, these methods primarily optimize individual models for specific demands, often requiring architectural redesigns or task-specific modifications, making the storage and deployment of multiple models costly. Consequently, there is an urgent need for strategies that can enable cost-effective, multi-task adaptation.

Parameter-efficient fine-tuning. Complementary to model compression, parameter-efficient fine-tuning provides a scalable path for personalization without modifying the pre-trained foundation model. Most prominently, Low-Rank Adaptation (LoRA) [9] injects trainable decomposition matrices while freezing backbone weights, enabling efficient multi-task adaptation via lightweight adapter swapping. Building upon this foundation, LoRA-Pro [20] refines the optimization of low-rank gradients to better approximate full-parameter updates; LoRAC [21] facilitates stable continual learning through QR decomposition; and HyperAT [22] leverages a HyperNetwork to generate defense-specific adapters. Recently, SG-LoRA [23] further explores semantic-guided LoRA parameter generation for task-specific adaptation. Moreover, Layerlink [24] bridges large models to detection through conduct adapters, and MA-FSAR [25] implements multimodal adaptation for action-related semantics. While these strategies provide a flexible alternative to model replication, they often rely on static low-rank structures and heuristic hyperparameter tuning, limiting their ability to precisely cater to varying requirements across different networks and tasks. To this end, we propose DPLoRA, a distillation-enhanced framework that adaptively composes multiple low-rank updates in parallel. By aligning student updates with the teacher's gradient subspace, DPLoRA enables more expressive and high-fidelity personalization while preserving the efficiency of the pretrained backbone.

3. Method

This section begins by observing that the diversity among fine-tuned SDMs arises from distinct domain-specific update subspaces formed during task adaptation. To efficiently approximate these subspaces without the overhead of full fine-tuning, we propose **DPLoRA**, a framework that leverages parallel LoRA branches equipped with learnable importance factors to automatically identify the corresponding subspaces of various SDMs. Additionally, we introduce an efficient reparameterization technique to further optimize inference performance. Finally, we provide a comprehensive overview of the DPLoRA distillation workflow.

3.1. Preliminary concepts

Most popular SDMs in the community are fine-tuned from shared pretrained large models (PLMs). This suggests that these SDMs share a common knowledge base, while their differences primarily emerge

from domain-specific subspaces induced by domain adaptation, endowing each model with domain-specific knowledge. To better understand how such subspaces can be efficiently captured, we revisit the paradigms of full fine-tuning (FFT) and LoRA [9].

Full fine-tuning (FFT) is a training process in which all model parameters are updated based on the computed gradients. Specifically, let $\theta_0 \in \mathbb{R}^{m \times n}$ be the pretrained weight matrix, where m and n denote the output and input dimensions, respectively. FFT updates it to $\theta = \theta_0 + \Delta\theta$ via gradient descent step $d\theta \propto -\partial\mathcal{L}/\partial\theta := -g_{\text{fft}}$ to minimize the corresponding loss $\mathcal{L}$. The continuous change in loss can be bounded as:

$$d\mathcal{L} = \left\langle \frac{\partial\mathcal{L}}{\partial\theta}, d\theta \right\rangle_F \propto -\left\| \frac{\partial\mathcal{L}}{\partial\theta} \right\|_F^2 \leq 0 \quad (1)$$

where $\langle \cdot \rangle_F$ denotes the Frobenius inner product, indicating the loss monotonically decreases. After training, the final weight matrix θ retains the same dimensions as the initial matrix θ_0 , thus incurring substantial storage overhead.

Fortunately, [26] demonstrate the existence of a low intrinsic dimension in learned over-parameterized models. Inspired by this finding, LoRA [9] hypothesizes that weight updates during FFT lie in a low-rank subspace, i.e., $\text{rank}(g_{\text{fft}}) \ll \min(m, n)$. To represent this, LoRA freezes the pretrained weights and expresses these updates as the product of two low-rank matrices:

$$\theta^{m \times n} = \theta_0^{m \times n} + \Delta\theta = \theta_0^{m \times n} + AB^T, \quad (2)$$

where $A \in \mathbb{R}^{m \times d}$ and $B \in \mathbb{R}^{n \times d}$ (d denotes the intrinsic rank, where $d \ll \min(m, n)$). During optimization, applying the chain rule yields the updates for A and B :

$$dA = -\frac{\partial\mathcal{L}}{\partial A} = -g_A, \quad dB^T = -\frac{\partial\mathcal{L}}{\partial B^T} = -A^T g_{\text{fft}} = -g_B, \quad (3)$$

which is equivalent to optimizing the full matrix θ as:

$$d\theta = \frac{\partial\theta}{\partial A} dA + \frac{\partial\theta}{\partial B^T} dB^T = -(g_A B^T + A g_B) = -g_{\text{LoRA}}. \quad (4)$$

According to Eq. (4), the optimization process of LoRA is equivalent to FFT using a restricted low-rank gradient g_{LoRA} , where $\text{rank}(g_{\text{LoRA}}) \leq 2d$.⁴ By adjusting the adapter rank d , LoRA can approximate the gradient space of FFT and thereby achieve comparable performance.

Following the above theory, the transformation from a base model to a target fine-tuned SDM can be characterized by an ideal domain-specific gradient trajectory, denoted as g_{sdm} . Capturing this optimal subspace via a static LoRA introduces a hyperparameter trade-off: (i) If $\text{rank}(g_{\text{LoRA}}) < \text{rank}(g_{\text{sdm}})$, the representational capacity of LoRA's gradient is insufficient to capture the necessary domain knowledge, leading to degraded performance; (ii) If $\text{rank}(g_{\text{LoRA}}) > \text{rank}(g_{\text{sdm}})$, redundant parameters irrelevant to domain adaptation are introduced, increasing computational overhead and potentially causing overfitting.

3.2. Dynamic LoRA branches

The limitations of static LoRA inspire us to find a more flexible approach to identify the ideal domain gradient evolution of SDMs while retaining the advantages of subspace optimization. To circumvent the reliance on a manually-tuned hyperparameter d , we aim to employ a learnable parameter to dynamically modulate its gradient space, allowing automatic alignment with the domain gradient subspace of each layer.

We consider the use of k parallel LoRA branches, each with a rank r , which can be formulated as:

$$\begin{aligned} \theta^{m \times n} &= \theta_0^{m \times n} + \sum_{i=1}^k \text{LoRA}^i(\theta_c^r) \\ &= \theta_0^{m \times n} + \sum_{i=1}^k a_i b_i^T \end{aligned} \quad (5)$$

⁴ Proof in Appendix A.2

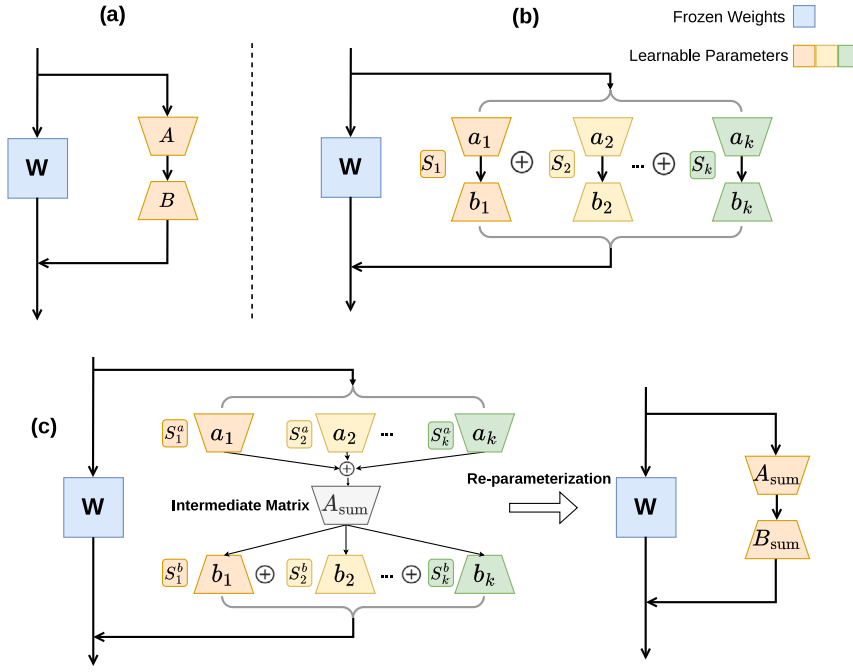


Fig. 2. Structural comparison: (a) Original LoRA. (b) NPLoRA with Importance Factors. (c) DPLoRA with Learnable Importance Factors and Re-Parameterization.

where $a_i \in \mathbb{R}^{m \times r}$, $b_i \in \mathbb{R}^{n \times r}$ (k denotes the total number of parallel branches, and r represents the base rank of each branch). The rank of the resulting equivalent gradient is upper-bounded by $2kr$.⁵ This method, termed **PLoRA**, shifts control of the effective gradient space from the single large rank d to the number of branches k .

However, simply summing parallel branches still constrains the gradient capacity rigidly by the fixed branch number k . Inspired by AdaLoRA [27], which prunes less important components to achieve dynamic rank, we propose **Naive Parallel LoRA** (NPLoRA, Fig. 2(b)). NPLoRA assigns each branch a learnable importance factor $s_i \in [0, 1]$. If the importance factor is smaller than $\frac{1}{2k}$,⁶ this branch is considered negligible and is subsequently pruned. To maintain end-to-end differentiability during training, this threshold is implemented as a forward masking strategy, allowing gradients to still flow through the importance factors. Mathematically, NPLoRA can be formally expressed as:

$$\theta_{nplora}^{m \times n} = \theta_0^{m \times n} + \sum_{i=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} s_i a_i b_i^T, \quad (6)$$

where $a_i \in \mathbb{R}^{m \times r}$, $b_i \in \mathbb{R}^{n \times r}$ and $\mathbf{1}_{\{s_i \geq \frac{1}{2k}\}}$ is the indicator function that retains branches exceeding the threshold $\frac{1}{2k}$ while pruning those below it. By Eq. (6), we derive the equivalent gradient of FFT by NPLoRA⁷:

$$g_{nplora} = \sum_{i=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} (s_i g_{a_i} b_i^T + s_i a_i g_{b_i} + a_i b_i^T g_{s_i}), \quad (7)$$

where g_{a_i} , g_{b_i} , and g_{s_i} denote the gradients with respect to a_i , b_i , and s_i . The rank of g_{nplora} is bounded by (see footnote 7):

$$\text{rank}(g_{nplora}) \leq \sum_{i=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} \times 3r. \quad (8)$$

Thus, NPLoRA dynamically adjusts its gradient dimensionality via learnable coefficients s_i , enabling it to converge to the optimal domain-specific gradient subspace of each layer while eliminating redundant parameters.

3.3. DPLoRA: Exploring vast gradients via re-parameterization

While NPLoRA achieves dynamic rank adjustment, its parallel structure results in significantly higher memory and computational costs during inference compared to a single LoRA branch. RepVGG [28] addresses a similar challenge using structural re-parameterization, which enables training of multiple parallel VGG branches (i.e., a_1, a_2, a_3) while merging them into a single parameter matrix at inference time, following the principle $a_1x + a_2x + a_3x = (a_1 + a_2 + a_3)x$. However, this design is non-trivial for NPLoRA, where each branch is a matrix product $a_i b_i^T$, since:

$$\sum_i^k a_i b_i^T x \neq \left(\sum_i^k a_i \right) \left(\sum_i^k b_i^T \right) x. \quad (9)$$

To address this limitation, we propose an elegant re-parameterization strategy: the down-projection matrices ($a_1, \dots, a_k$) are first aggregated into an intermediate matrix A_{sum} , and subsequently combined with the up-projection matrices ($b_1, \dots, b_k$) into B_{sum} , forming a single effective branch (Fig. 2(c)). Moreover, we assign separate learnable weights s_i and s_j to the down- and up-projections, respectively, reflecting the intuition that their contributions to the gradient subspace are not necessarily symmetric. Formally, the re-parameterization can be expressed as:

$$\begin{aligned} \theta_{dplora}^{m \times n} &= \theta_0^{m \times n} + \sum_{j=1}^k \mathbf{1}_{\{s_j \geq \frac{1}{2k}\}} s_j \left(\sum_{i=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} s_i a_i \right) b_j^T \\ &= \theta_0^{m \times n} + \underbrace{\left(\sum_{i=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} s_i a_i \right)}_{A_{\text{sum}} \in \mathbb{R}^{m \times r}} \cdot \underbrace{\left(\sum_{j=1}^k \mathbf{1}_{\{s_j \geq \frac{1}{2k}\}} s_j b_j^T \right)}_{B_{\text{sum}}^T \in \mathbb{R}^{r \times n}} \\ &= \theta_0^{m \times n} + A_{\text{sum}} \cdot B_{\text{sum}}^T \end{aligned} \quad (10)$$

where $a_i \in \mathbb{R}^{m \times r}$, $b_j \in \mathbb{R}^{n \times r}$, $A_{\text{sum}} \in \mathbb{R}^{m \times r}$ and $B_{\text{sum}} \in \mathbb{R}^{n \times r}$.

We refer to this factorized design as **DPLoRA**. Importantly, during the forward pass, computing A_{sum} and B_{sum} allows the network to behave exactly like a single LoRA branch with rank r , eliminating the inference overhead of parallelism. DPLoRA updates a_i , b_i , s_i , s_j with gradients g_{a_i} , g_{b_i} , g_{s_i} , g_{s_j} and is equivalent to performing full fine-tuning

⁵ Derivation provided in Appendix A.3 PLoRA

⁶ Details provided in Proposition 2 Justification for Threshold

⁷ Detailed derivation in NPLoRA

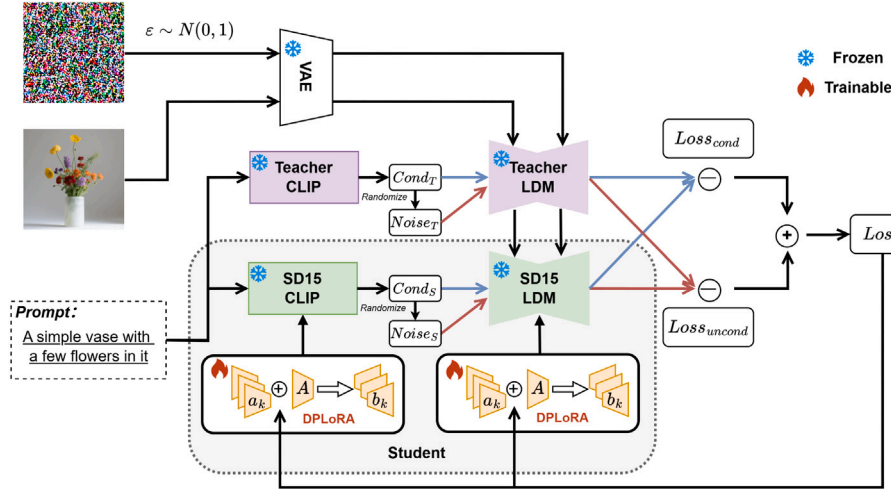


Fig. 3. Knowledge distillation pipeline via DPLoRA.

on θ via the following highly expressive cross-gradient⁸:

$$g_{dplora} = \sum_{i=1}^k \sum_{j=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} \mathbf{1}_{\{s_j \geq \frac{1}{2k}\}} \left(s_j (s_i g_{a_i} + a_i g_{s_i}) b_j^\top + s_i a_i (s_j g_{b_j} + b_j^\top g_{s_j}) \right) \quad (11)$$

with rank (see footnote):

$$\text{rank}(g_{dplora}) \leq \sum_{i=1}^k \sum_{j=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} \mathbf{1}_{\{s_j \geq \frac{1}{2k}\}} \times 4r \leq 4k^2 r. \quad (12)$$

Notably, we observe that under fixed number of trainable parameters ($kr = d$), the upper bound of DPLoRA's gradient rank is $4k^2 r$, compared to $3kr$ for NPLoRA and $2kr$ for LoRA. This demonstrates that while DPLoRA maintains a lightweight rank r during forward inference, it explores an expansive $O(k^2)$ gradient subspace during backward optimization. This drastically increases the likelihood of identifying the optimal domain-specific subspace for each SDM's layer without extra inference cost.

3.4. Distillation training with DPLoRA

To guide DPLoRA in approximating the intrinsic subspace of target SDMs, we adopt a teacher–student distillation paradigm where the finetuned SDM serves as the teacher, and the base model together with the layer-wise injected DPLoRA modules act as the student. During distillation (see Fig. 3), the parameters of the base model are frozen, and only the DPLoRA modules are updated. Specifically, DPLoRA is injected into both CLIP and LDM components, while the VAE component remains unchanged, as it only provides pixel-to-latent space conversion and contributes little to domain adaptation. To enhance teacher guidance, random text hidden states are sampled from the real input with an identical shape, serving as unconditional input. This enables the student to unconditionally and conditionally imitate the teacher's latent trajectories. The distillation training objective is formulated as the minimization of the mean squared error (MSE) between the teacher output and the student output. By minimizing this loss, DPLoRA leverages its expansive gradient exploration space to swiftly converge to the domain-specific update subspace of the teacher while maintaining unparalleled parameter efficiency and flexibility. The detailed algorithm is presented in Algorithm 1.

Algorithm 1 SDM knowledge distillation via DPLoRA

Require: Teacher θ_T , Base θ_{base} , Dataset D , DPLoRA δ

Ensure: Optimized δ

- 1: $\theta_S \leftarrow \text{Inject}_{\text{CLIP+LDM}}(\theta_{base}, \delta)$; freeze θ_T and θ_{base}
- 2: **while not converged do**
- 3: Sample $(x, y) \sim D$, $\epsilon \sim \mathcal{N}(0, \mathbf{I})$, $t \sim \mathcal{U}(1, T)$
- 4: $z_t \leftarrow \text{VAE}(x) + \epsilon$
- 5: $c_{\text{cond}} = \text{CLIP}(y)$, $c_{\text{uncond}} = \text{Randomize}(c_{\text{cond}})$
- 6: $C = \{c_{\text{cond}}, c_{\text{uncond}}\}$
- 7: $\mathcal{L}_{\text{distill}} = \sum_{c \in C} \|\theta_T(z_t, t, c) - \theta_S(z_t, t, c; \delta)\|_2^2$
- 8: Update δ by minimizing $\mathcal{L}_{\text{distill}}$
- 9: **end while**
- 10: **return** δ

4. Experiments

4.1. Setup

Implementation details. We implement our framework using the Diffusers library. All training are conducted on 8 NVIDIA A100 GPUs using CUDA 12.4 and bfloat16 mixed precision for computational efficiency. The model is trained for 70,000 steps with a batch size of 16 via the AdamW optimizer. We set the learning rates to 1×10^{-5} for the U-Net and 1×10^{-6} for the text encoder. During training, input images are resized to 512×512 and normalized to the range $[-1, 1]$. We utilize the LAION-220k dataset⁹ for training and evaluate our approach using prompts from the MS-COCO-30k [29].

Metrics. To evaluate generative performance for Stable Diffusion models, we use the widely adopted Fréchet Inception Distance (FID) [30], which quantifies the distributional discrepancy between generated and target data. Additionally, we use the CLIP Score (CLIP) [31] to evaluate both the quality of generated images and their alignment with the corresponding prompts. For large language models (LLMs), we evaluate performance using Pass@1 on the HumanEval dataset [32], Accuracy on the GSM8K benchmark [33], and GPT Scores on MT-Bench [34]. To assess practical deployment efficiency, we also consider the number of parameters (#Params) and latency (Latency) as metrics for comparing different methods.

⁸ Detailed derivation in Appendix A.4 DPLoRA

⁹ <https://huggingface.co/datasets/laion/220k-GPT4Vision-captions-from-LIVIS>

Table 1

Quantitative comparison among baselines on image generation tasks. The results closest and second-closest to the Target are highlighted in **bold** and underlined, respectively.

Method	Realistic	Yesmix		DisneyPixar			
		FID	CLIP	FID	CLIP	FID	CLIP
Target	–	0.00	0.3051	0.00	0.2751	0.00	0.2709
SD15 (Base)	–	6.70	0.2967	15.05	0.2967	24.34	0.2967
+LoRA [9]	Rank32	4.45	0.3023	8.14	0.2964	12.34	0.2919
	Rank64	3.65	0.3017	7.63	0.2952	11.23	0.2906
	Rank128	3.61	0.3026	7.18	0.2936	<u>10.39</u>	0.2895
+AdaLoRA [27]	Rank128 → 32	3.99	0.3020	7.65	0.2941	10.84	0.2901
+LoRAPro [20]	Rank128	3.56	0.3033	7.05	<u>0.2821</u>	10.66	<u>0.2761</u>
+DPLoRA	Rank32+Branch8	<u>3.60</u>	<u>0.3031</u>	7.03	0.2781	10.22	0.2736
BKSDM [13]	Base	5.16	0.2987	11.73	0.2637	12.15	0.2747
	Small	10.81	0.2802	19.33	0.2547	16.78	0.2528
	Tiny	25.83	0.2746	20.80	0.2517	19.12	0.2447

Baseline methods. We benchmark DPLoRA against a diverse set of state-of-the-art baselines, ranging from full fine-tuning to various parameter-efficient fine-tuning approaches, including LoRA [9], rsLoRA [35], PiSSA [36], LoRA+ [37], DoRA [38], AdaLoRA [27], LoRA-GA [39], and LoRA-Pro [20]. For image generation tasks, we further compare with the representative SDM distillation method BKSDM [13].

Target teacher models. To demonstrate the applicability of our method, we select three widely used SD15-based fine-tuned SDMs from Hugging Face as distillation targets: RealisticVisionV4,¹⁰ YesmixV35,¹¹ and DisneyPixar.¹² Each model represents a distinct visual style, covering diverse domain-specific knowledge.

4.2. Comparison

Quantitative results on image generation. For Stable Diffusion models, quantitative evaluation is conducted on MSCOCO-30k, where DPLoRA uses a rank of 32 and is initialized with 8 branches per layer. Semantic similarity is measured using the CLIP Score, where a well-trained student model is expected to exhibit a CLIP score close to that of the teacher (target) model, indicating a comparable understanding of the input prompts. As shown in Table 1, DPLoRA consistently outperforms LoRA [9], AdaLoRA [27], and BKSDM [13], achieving performance nearly identical to the target models. Specifically, DPLoRA achieves an FID of 7.03 on Yesmix and 10.22 on DisneyPixar, both surpassing LoRA-128. While LoRA [9] demonstrates improved performance with increasing ranks on Yesmix and DisneyPixar, its convergence behavior on the Realistic model highlights the inherent challenges of manual rank tuning. DPLoRA addresses this limitation by automatically identifying an optimal gradient subspace through its dynamic parallel-merging structure. Crucially, although LoRAPro [20] achieves marginally better FID and CLIP scores on the Realistic model, DPLoRA maintains a key advantage in practical deployment: it significantly reduces inference and storage costs by compressing the final rank to 32, whereas LoRAPro relies on a higher rank of 128. These results highlight DPLoRA’s capability to effectively capture complex domain-specific gradient structures in SDMs with minimal storage and inference overhead, making it particularly suitable for multi-model deployment.

Quantitative results on language tasks. For Large Language Models, following the experimental setup of LoRA-Pro [20] for the selection of training and evaluation datasets, we conduct quantitative evaluation on MT-Bench, GSM8K, and HumanEval benchmarks using the Llama-2-7B model fine-tuned with different methods to assess dialogue, mathematical reasoning, and code generation capabilities. The model is trained

Table 2

Quantitative comparison of different methods on LLM benchmarks. The best and second-best results are highlighted in **bold** and underlined, respectively. R denotes the rank, and B denotes the number of branches.

Method	MT-Bench	GSM8K	HumanEval
Full FT	5.30 ± 0.11	59.36 ± 0.85	35.31 ± 2.13
LoRA [9]	5.61 ± 0.10	42.08 ± 0.04	14.76 ± 0.17
PiSSA [36]	5.30 ± 0.02	44.54 ± 0.27	16.02 ± 0.78
rsLoRA [35]	5.25 ± 0.03	45.62 ± 0.10	16.01 ± 0.79
LoRA+ [37]	5.71 ± 0.08	52.11 ± 0.62	18.17 ± 0.52
DoRA [38]	<u>5.97 ± 0.02</u>	53.07 ± 0.75	19.75 ± 0.41
AdaLoRA [27]	<u>5.57 ± 0.05</u>	50.72 ± 1.39	17.80 ± 0.44
LoRA-GA (R32) [39]	5.79 ± 0.09	55.12 ± 0.30	20.18 ± 0.19
LoRA-GA (R128)	6.13 ± 0.07	55.07 ± 0.18	23.05 ± 0.37
LoRA-Pro (R32) [20]	5.57 ± 0.51	57.97 ± 0.50	26.63 ± 0.35
LoRA-Pro (R128)	5.67 ± 0.11	61.08 ± 0.19	30.28 ± 0.93
DPLoRA (R8 B4)	5.68 ± 0.05	56.18 ± 0.38	26.83 ± 1.22
DPLoRA (R32 B4)	5.58 ± 0.03	58.96 ± 0.64	30.73 ± 0.74
DPLoRA (R32 B8)	5.52 ± 0.14	<u>59.46 ± 0.76</u>	32.11 ± 0.93
DPLoRA (R64 B4)	5.43 ± 0.11	59.39 ± 0.55	<u>35.28 ± 2.07</u>

on subsets of WizardLM (52k), MetaMathQA (100k), and CodeFeedback (100k), enabling a comprehensive evaluation of DPLoRA’s scalability and domain-specific adaptability. As shown in Table 2, on HumanEval, DPLoRA (R64 B4) achieves a Pass@1 score of 35.28, nearly matching Full FT (35.31) and significantly outperforming LoRA-Pro (R128) and LoRA-GA (R128). On GSM8K, DPLoRA (R32 B8) attains an accuracy of 59.46, surpassing the Full FT baseline (59.36). On MT-Bench, however, DPLoRA underperforms relative to other PEFT methods, exhibiting a trend consistent with the unexpected underperformance of Full FT (5.30). We attribute this to the distribution mismatch between the WizardLM training data and the MT-Bench evaluation set. The high representational capacity of Full FT leads to overfitting on the training distribution, thereby degrading generalization on MT-Bench. As DPLoRA and LoRA-Pro follow optimization trajectories closer to Full FT, their performance on MT-Bench is correspondingly affected. Overall, DPLoRA demonstrates strong reasoning and coding capabilities, effectively narrowing the gap to Full FT.

Qualitative. The qualitative comparison evaluates the visual effects of DPLoRA, LoRA, and BKSDM-Base across three styles: Yesmix, Realistic, and Disney. Fig. 4 shows the visual results, highlighting the effectiveness of our knowledge distillation process. DPLoRA demonstrates high fidelity to the input prompts, while BKSDM shows noticeable distortions in facial features. Major details, such as hairstyle, outfits, compositional consistency, and overall style, are preserved in DPLoRA compared to LoRA and BKSDM. Additionally, as shown in the third row, minor details such as eye direction, brow color, and moustache shape are captured by DPLoRA.

¹⁰ https://huggingface.co/SG161222/Realistic_Vision_V4.0_noVAE

¹¹ <https://huggingface.co/stablediffusionapi/yesmix-v35>

¹² <https://huggingface.co/stablediffusionapi/disneypixar>

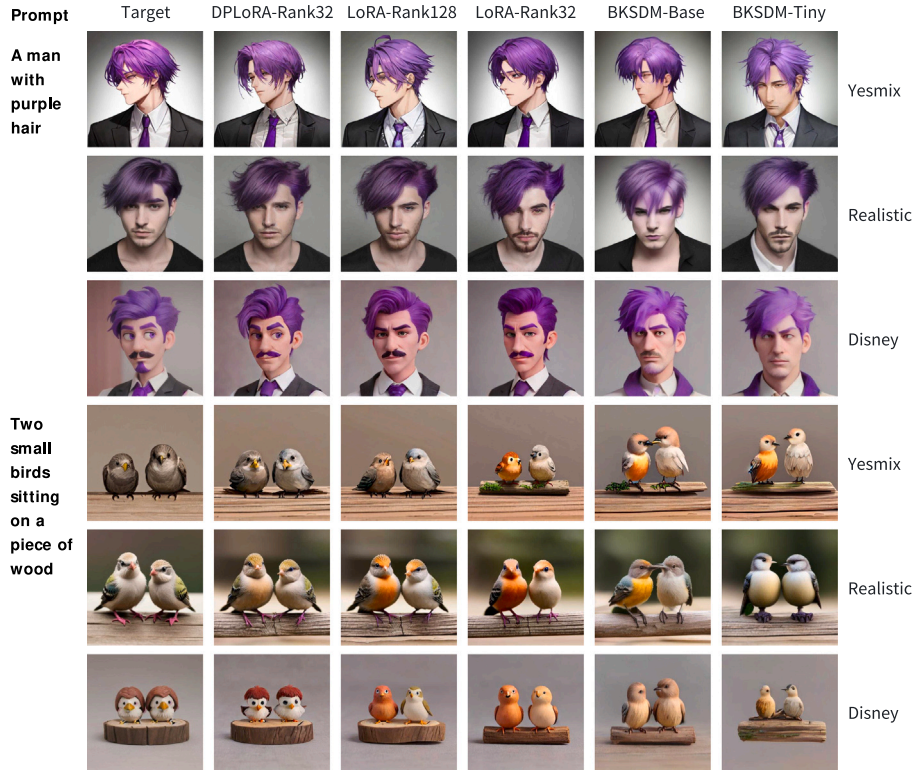


Fig. 4. Generated images of different distilled models.

Table 3

Parameter (B) and latency (s) for executing an image generation workflow with n SDMs. R32 indicates rank=32.

Methods	1 Model		3 Models		5 Models		10 Models	
	Params B	Latency s	Params B	Latency s	Params B	Latency s	Params B	Latency s
Full	1.07	1.71	3.20	7.76	5.33	14.68	10.66	31.64
LoRA-R32	1.10	1.95	1.16	5.62	1.22	9.36	1.37	18.97
DPLoRA-R32	1.10	1.95	1.16	5.62	1.22	9.36	1.37	18.97
BKSDM-Base	0.79	0.90	2.36	6.10	3.93	11.69	7.86	27.01

Table 4

Total runtime for completing all requests under different parallel deployment scales, where each deployed model is assigned 200 requests.

Parallel deployment scale	FFT	BKSDM	DPLoRA-R32
12 Models	23.4 min	17.8 min	17.6 min
15 Models	30.6 min	23.4 min	21.7 min
18 Models	44.7 min	33.0 min	27.4 min

4.3. Performance

Serial model-switching efficiency. To demonstrate the advantages of DPLoRA regarding memory and inference speed in low-resource settings, we deploy multiple SDMs on a single RTX 3090 GPU. By varying the number of SDMs in use, we compare the total number of parameters and inference latency across different methods. Latency is computed by generating five images, which involves alternating between models, while memory requirements are represented by the number of parameters. As shown in Table 3, DPLoRA offers a significant advantage over other methods in a single-GPU setup. Full model configurations and BKSDM [13] suffer from heavyweight architectures that incur substantial model-switching overhead, resulting in high latency when multiple models are required. In contrast, DPLoRA enables near-instantaneous switching by simply swapping lightweight adapter weights.

Parallel deployment efficiency. Beyond parameter size and model-switching latency, we further evaluate the end-to-end serving efficiency

in a practical parallel deployment scenario on a single NVIDIA A100 GPU. As shown in Table 4, each deployed model is assigned 200 generation requests, and we report the time required to complete all requests under different deployment scales. Compared with FFT, DPLoRA consistently reduces the total runtime from 23.4, 30.6, and 44.7 min to 17.6, 21.7, and 27.4 min when deploying 12, 15, and 18 models, respectively. This indicates that DPLoRA maintains its advantage not only in serial model switching but also in batch and concurrent generation settings. This advantage mainly comes from sharing the base diffusion model across multiple lightweight task-specific LoRA modules, thereby reducing redundant model memory and improving the effective serving capacity under the same GPU budget.

4.4. Ablation study

Branch dynamism. The pruning mechanism facilitates the dynamic adaptability of DPLoRA and significantly improves its performance, as demonstrated in Sections 4.2 and 4.3. In this section, we provide further empirical validation regarding the efficacy of the pruning mechanism by visualizing the evolution of active branches during the training process. Specifically, we randomly select four layers – two from CLIP and two from UNet – and plot the changes in the number of branches over training steps in Fig. 5 (left). Additionally, the final number of branches for each layer after the training process is summarized in Fig. 5 (right). As illustrated, the number of active branches gradually decreases throughout training, with the pruning mechanism exhibiting

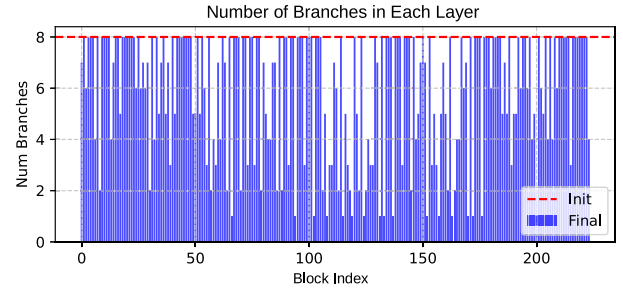
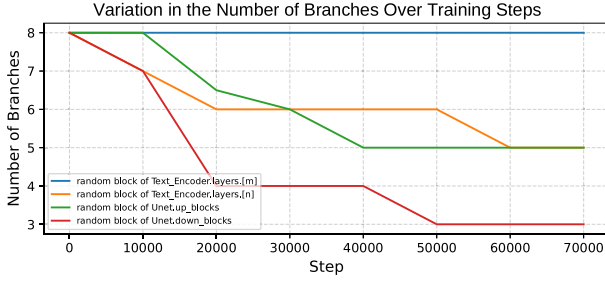


Fig. 5. (Left) Randomly selected layers show branch pruning. (Right) The number of branches in each layer after training.

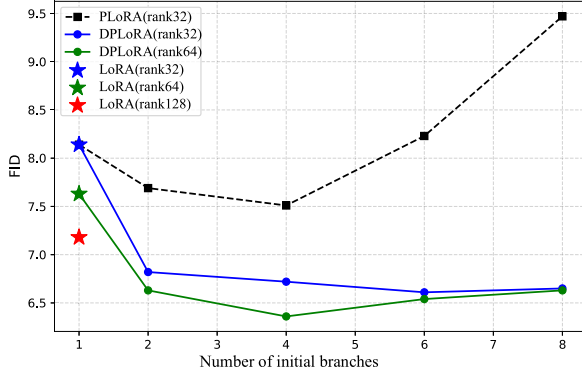


Fig. 6. Line plots showing how performance varies along the rank and branch number and the effect of importance factors.

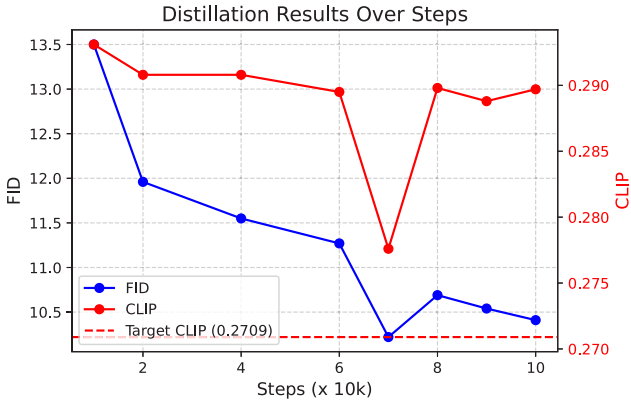


Fig. 7. Convergence steps measured by FID and CLIP.

heterogeneous behavior across different layers. This provides strong empirical evidence that DPLoRA successfully identifies the varying rank requirements of different layers for domain-specific knowledge acquisition.

Rank, branch number, and importance factors. We aim to investigate how different factors (rank, initial branch number, and the presence of importance factors) affect DPLoRA's performance and further validate the effectiveness of the proposed re-parameterization technique. We evaluate different configurations of rank and branch numbers using FID as the primary evaluation metric. NPLoRA serves as a baseline to assess the impact of the importance factors and the necessity of re-parameterization. As illustrated in Fig. 6, NPLoRA achieves its peak performance with $k = 4$ branches, whereas DPLoRA (for both $r = 32$ and $r = 64$) plateaus as the number of branches increases, demonstrating the robustness provided by the importance factors and re-parameterization. Notably, when constrained to the same number of trainable parameters

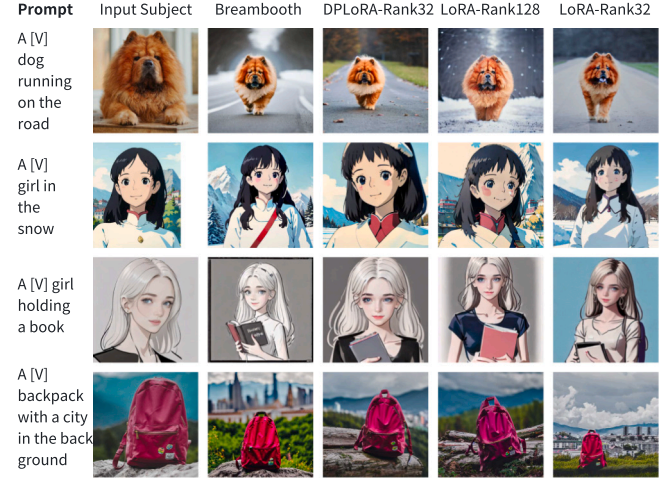


Fig. 8. DPLoRA can also be applied to full model fine-tuning tasks, such as DreamBooth.

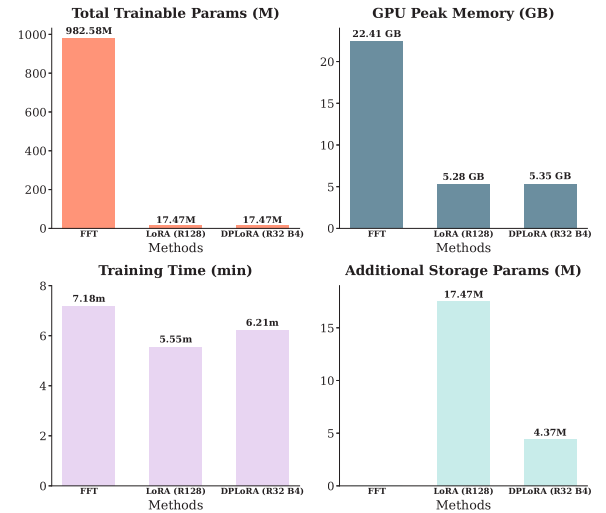


Fig. 9. Efficiency comparison on DreamBooth across FFT, LoRA, and DPLoRA.

as LoRA and NPLoRA, DPLoRA consistently exhibits superior performance. This aligns with our theoretical assertion that DPLoRA benefits from an expanded gradient subspace, thereby possessing enhanced representational capacity. Additionally, we observe that the DPLoRA configuration with $r = 32, k = 8$ does not fully match the performance of the $r = 64, k = 4$ setup. We attribute this minor discrepancy to initialization variance, as the resulting difference in FID remains negligible.

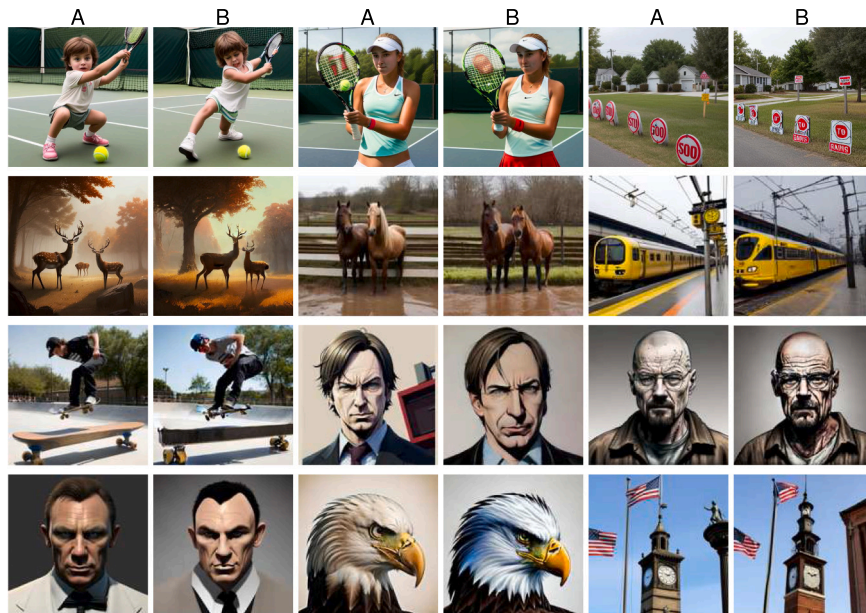


Fig. 10. More distillation results with selected poor and good examples. Column A indicates the generation result of original SDM, Column B indicates the generation result of our method.

Convergence. We also explore the optimal training steps for DPLoRA by recording changes in the performance metrics during the training process. For this analysis, we utilize a DPLoRA configuration with $r = 32$ and $k = 8$, as illustrated in Fig. 7. In the plot, a CLIP score asymptotically approaching the dashed line (representing the teacher’s performance) indicates successful semantic alignment, while the FID score attains convergence upon reaching its minimum value. The results demonstrate that the model achieves optimal performance at approximately 70,000 training steps, after which the metrics stabilize.

4.5. Extension to personalized generation

Beyond its application in knowledge distillation, DPLoRA is also well-suited for fine-tuning and creating personalized models. As illustrated in Fig. 8, we present a visual comparison among DreamBooth [12], DPLoRA, and LoRA to highlight these use cases. The generated images demonstrate that DPLoRA effectively captures fine-grained subject identity and produces high-fidelity images on par with those generated by DreamBooth. Crucially, this competitive visual performance is achieved with minimal computational overhead. As shown in Fig. 9, DPLoRA significantly outperforms Full Fine-Tuning (FFT) in efficiency on the DreamBooth task, reducing trainable parameters by over 98% (17.47M vs. 982.58M) and cutting peak memory from 22.41 GB to 5.35 GB. DPLoRA also achieves a faster training time of 6.21 min compared to FFT (7.18 min), while providing a significant 75% reduction in storage requirements (4.37M vs. 17.47M) compared to standard LoRA (Rank 128), making it highly effective for resource-constrained deployment.

4.6. More successes and failure cases of DPLoRA

Fig. 10 presents a visual comparison between the teacher model (A) and the proposed DPLoRA (B). Overall, DPLoRA effectively captures both the stylistic details and the primary subject characteristics of the teacher model. However, minor artifacts are still observable in column B, particularly in regions such as the child’s limbs, the tennis ball, and the road sign. These artifacts are partly related to the LoRA-style adaptation used by DPLoRA, where task-specific updates are represented by lightweight low-rank branches rather than dense full-parameter updates. This representation is largely sufficient for transferring dominant

style and subject information, but small differences may remain in local regions that require precise structural modeling. For example, the child’s limbs require accurate anatomical consistency under complex poses, small objects such as the tennis ball are sensitive to localization and boundary preservation, and road signs involve text-like or symbolic patterns that are difficult to render accurately in text-to-image diffusion models. The remaining errors therefore mainly reflect the combined difficulty of low-rank adaptation and diffusion-based generation in modeling fine-grained structural, spatial, and symbolic details.

5. Limitations and future work

While our current evaluation focuses on fine-tuned models derived from SD 1.5, the proposed DPLoRA framework is inherently architecture-agnostic, as it operates at the level of parameter-efficient adaptation rather than relying on model-specific design. This ensures compatibility with diverse model architectures. Notably, we have empirically verified the feasibility of extending DPLoRA to large language models (LLMs), where our method demonstrates consistent effectiveness in capturing domain-specific knowledge, suggesting its broader applicability beyond diffusion models. However, a significant limitation of the current framework is the requirement for one-to-one distillation for each target model, and it does not yet explore how to adapt a single module to multiple target models. Moreover, the model occasionally struggles to capture extremely fine-grained structural details in complex visual compositions.

In the future, we will focus on the following directions. First, we plan to extend DPLoRA to more advanced generative architectures, such as SD 3, to further validate its generalization capability. Second, we aim to explore a unified DPLoRA formulation that can simultaneously approximate multiple fine-tuned models within a single framework, further improving efficiency for large-scale deployment. Finally, we aim to improve fine-grained structural fidelity by exploring more robust distillation objectives.

6. Conclusion

In this work, we address a critical bottleneck in the deployment of generative AI: the prohibitive memory footprint and switching overhead incurred when maintaining multiple independent SDMs for diverse stylistic requirements. To this end, we propose DPLoRA, an

adaptation and distillation framework that extracts domain-specific knowledge into lightweight adapters while sharing a unified backbone, thereby enabling efficient multi-domain generation under limited hardware resources. Unlike conventional adaptation methods such as LoRA, which rely on manual rank configurations, DPLoRA introduces a dynamic parallel branching mechanism with structural reparameterization. This design allows the model to automatically explore a richer $O(k^2)$ gradient subspace during optimization, facilitating the capture of complex domain-specific knowledge. At inference time, these parallel branches can be mathematically merged into a single standard LoRA module, incurring zero additional storage or computational overhead compared with vanilla LoRA. Despite its effectiveness, DPLoRA faces challenges in fine-grained fidelity and one-to-one distillation overhead. Future work will focus on addressing these through robust training objectives, unified multi-target distillation, and extending the framework to next-generation architectures such as Stable Diffusion 3. In summary, this work provides a practical solution for deploying diverse generative models in resource-constrained environments, effectively bridging the gap between cutting-edge research and real-world generative AI applications.

CRedit authorship contribution statement

Weibin Zeng: Writing – original draft, Visualization, Methodology. **Yan Kong:** Writing – original draft, Resources. **Fuzhang Wu:** Writing – review & editing, Writing – original draft. **Youheng Ren:** Methodology, Investigation, Data curation. **Sicheng Shen:** Visualization, Methodology, Investigation. **Yuqing Fan:** Writing – review & editing, Writing – original draft. **Weiming Dong:** Writing – original draft, Methodology, Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by Major Project of ISCAS (ISCAS-ZD-E6GG3102).

Appendix

A.1. Notation

In Table A.5, we detail the notations utilized in our paper.

Table A.5
Description of notations used in the paper.

Notation	Description	Notation	Description
s_i, s_j	Importance factors of a LoRA branch	$\mathbf{1}\{\cdot\}$	Indicator function used as a pruning mask
$a_i \in \mathbb{R}^{m \times r}, b_j \in \mathbb{R}^{n \times r}$	Low-rank factor matrices	$g, g_{\text{ftt}} = \frac{\partial \mathcal{L}}{\partial \theta} \in \mathbb{R}^{m \times n}$	Gradients of full fine-tuning
$g_{a_i} = \frac{\partial \mathcal{L}}{\partial a_i} \in \mathbb{R}^{m \times r}$	Gradient of matrix a_i	$g_{b_j} = \frac{\partial \mathcal{L}}{\partial b_j} \in \mathbb{R}^{n \times r}$	Gradient of matrix b_j
$g_{s_i} = \frac{\partial \mathcal{L}}{\partial s_i} \in \mathbb{R}$	Gradient of importance factor s_i	$g_{s_j} = \frac{\partial \mathcal{L}}{\partial s_j} \in \mathbb{R}$	Gradient of importance factor s_j
$d\mathcal{L}$	Differential of the loss function	$d\theta$	Differential of the weight matrix θ
$\ \cdot\ _F$	Frobenius norm	$\langle \cdot, \cdot \rangle_F$	Frobenius inner product
$\text{tr}(\cdot)$	Trace of a matrix		

A.2. Analysis of LoRA's equivalent gradient subspace

Full fine-tuning. According to [20], we calculate the relationship between changes in the loss and changes in the weights:

$$d\mathcal{L} = \left\langle \frac{\partial \mathcal{L}}{\partial \theta}, d\theta \right\rangle_F, \quad (\text{A.1})$$

where $d\mathcal{L}$ and $d\theta$ denote the infinitesimal changes of the loss $\mathcal{L}$ and the parameter θ , respectively, and $\langle \cdot, \cdot \rangle_F$ denotes the Frobenius inner product. To minimize the loss function, we typically choose the update direction as

$$d\theta = -\frac{\partial \mathcal{L}}{\partial \theta} = -g \quad (\text{omit the learning rate}), \quad (\text{A.2})$$

which results in: $d\mathcal{L} = -\left\| \frac{\partial \mathcal{L}}{\partial \theta} \right\|_F^2 \leq 0$

Low rank adaptation. LoRA performs optimization over a low-rank subspace by decomposing the update into:

$$\theta = \theta_0 + AB^\top, \quad (\text{A.3})$$

where $A \in \mathbb{R}^{m \times d}$, $B \in \mathbb{R}^{n \times d}$. The gradient descent of the loss is given by:

$$dL = \left\langle \frac{\partial \mathcal{L}}{\partial \theta}, d\theta \right\rangle_F = \left\langle \frac{\partial \mathcal{L}}{\partial \theta}, \frac{\partial \theta}{\partial A} dA + \frac{\partial \theta}{\partial B} dB \right\rangle_F. \quad (\text{A.4})$$

Using the chain rule:

$$\begin{aligned} d\mathcal{L} &= \left\langle \frac{\partial \mathcal{L}}{\partial \theta}, \frac{\partial \theta}{\partial A}, dA \right\rangle_F + \left\langle \frac{\partial \mathcal{L}}{\partial \theta}, \frac{\partial \theta}{\partial B}, dB \right\rangle_F \\ &= \left\langle \frac{\partial \mathcal{L}}{\partial A}, dA \right\rangle_F + \left\langle \frac{\partial \mathcal{L}}{\partial B}, dB \right\rangle_F \end{aligned} \quad (\text{A.5})$$

To minimize the loss function in LoRA, we set:

$$\begin{aligned} dA &= -\frac{\partial \mathcal{L}}{\partial A} = -\frac{\partial \mathcal{L}}{\partial \theta} \frac{\partial \theta}{\partial A} = -g B = -g_A \\ dB^\top &= -\frac{\partial \mathcal{L}}{\partial B^\top} = -\frac{\partial \mathcal{L}}{\partial \theta} \frac{\partial \theta}{\partial B^\top} = -A^\top g = -g_B \end{aligned} \quad (\text{A.6})$$

which yields $d\mathcal{L} = -\left\| \frac{\partial \mathcal{L}}{\partial A} \right\|_F^2 - \left\| \frac{\partial \mathcal{L}}{\partial B} \right\|_F^2 \leq 0$, aligning with our objective, and LoRA would update $d\theta$

$$d\theta = \frac{\partial \theta}{\partial A} dA + \frac{\partial \theta}{\partial B^\top} dB^\top = -(g_A B^\top + A g_B). \quad (\text{A.7})$$

Eq. (A.7) reveals that LoRA optimization is equivalent to full fine-tuning using a low-rank gradient

$$\tilde{g}_{\text{lor}_a} = g_A B^\top + A g_B,$$

Proposition 1. If LoRA rank equals to d , the equivalent low-rank gradient is at most $2d$, $\text{rank}(\tilde{g}_{\text{lor}_a}) \leq 2d$.

Proof. We use subadditivity and composition properties of matrix rank:

$$\begin{aligned} \text{rank}(\tilde{g}_{\text{lor}_a}) &\leq \text{rank}(g_A B^\top) + \text{rank}(A g_B) \\ &\leq \min(\text{rank}(g_A), \text{rank}(B^\top)) + \min(\text{rank}(A), \text{rank}(g_B)) \end{aligned}$$

$$\begin{aligned} &\leq \min(\min(\text{rank}(g), \text{rank}(B), \text{rank}(B^\top)) \\ &\quad + \min(\text{rank}(A), \min(\text{rank}(A^\top), \text{rank}(g))) \\ &\leq d + d = 2d. \quad \square \end{aligned}$$

A.3. Analysis of nplora's equivalent gradient subspace

Parallel LoRA. Parallel LoRA introduces k LoRA branches, each with a rank of r :

$$W = W_0 + \sum_{i=1}^k a_i b_i^\top,$$

where $a_i \in \mathbb{R}^{m \times r}$, $b_i \in \mathbb{R}^{n \times r}$. Similar to LoRA, it update a_i , b_i with

$$da_i = -\frac{\partial \mathcal{L}}{\partial a_i} = -g b_i = -g_{a_i}; \quad db_i^\top = -\frac{\partial \mathcal{L}}{\partial b_i^\top} = -a_i^\top g = -g_{b_i} \quad (\text{A.8})$$

and its equals to update $d\theta$ as:

$$d\theta = -\sum_{i=1}^k (g_{a_i} b_i^\top + a_i g_{b_i}) = -\tilde{g}_{plora} \quad (\text{A.9})$$

Following Proposition 1, $\text{rank}(\tilde{g}_{plora})$ is upper bounded by $2kr$.

Naive parallel LoRA. Naive Parallel LoRA (NPLoRA) extends Parallel LoRA by introducing a learnable coefficient s_i for each branch:

$$\theta = \theta_0 + \sum_{i=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} s_i \cdot (a_i b_i^\top)$$

where $a_i \in \mathbb{R}^{m \times r}$, $b_i \in \mathbb{R}^{n \times r}$ and $s_i \in [0, 1]$. Differentiating yields gradients:

$$\begin{aligned} da_i &= -\frac{\partial \mathcal{L}}{\partial a_i} = -\frac{\partial \mathcal{L}}{\partial \theta} \frac{\partial \theta}{\partial a_i} = -\mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} s_i (g b_i) = -\mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} g_{a_i}; \\ db_i^\top &= -\frac{\partial \mathcal{L}}{\partial b_i^\top} = -\frac{\partial \mathcal{L}}{\partial \theta} \frac{\partial \theta}{\partial b_i^\top} = -\mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} s_i (a_i^\top g) = -\mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} g_{b_i}; \\ ds_i &= -\frac{\partial \mathcal{L}}{\partial s_i} = -\sum_{p=1}^m \sum_{q=1}^n \frac{\partial \mathcal{L}}{\partial \theta_{i,pq}} \frac{\partial \theta_{i,pq}}{\partial s_i} = -\mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} \langle g, a_i b_i^\top \rangle = -\mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} g_{s_i}; \end{aligned} \quad (\text{A.10})$$

¹³ Mathematically, NPLoRA equals to update the $d\theta$ as:

$$\begin{aligned} d\theta &= \sum_{i=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} \left(\frac{\partial \theta}{\partial a_i} da_i + \frac{\partial \theta}{\partial b_i^\top} db_i^\top + \frac{\partial \theta}{\partial s_i} ds_i \right) \\ &= -\sum_{i=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} (s_i g_{a_i} b_i^\top + s_i a_i g_{b_i} + a_i b_i^\top g_{s_i}) = -\tilde{g}_{nplora} \end{aligned} \quad (\text{A.11})$$

Proposition 2. For NPLoRA with branch-rank r and k branches, the equivalent low-rank gradient has rank at most

$$\text{rank}(\tilde{g}_{nplora}) \leq \sum_{i=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} \times 3r \quad (\text{A.12})$$

Proof. We use subadditivity and composition properties of matrix rank:

$$\begin{aligned} \text{rank}(\tilde{g}_{nplora}) &\leq \sum_{i=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} (\text{rank}(s_i g_{a_i} b_i^\top) + \text{rank}(s_i a_i g_{b_i}) + \text{rank}(a_i b_i^\top g_{s_i})) \\ &\leq \sum_{i=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} \times 3r \end{aligned}$$

At initialization, since $s_i \geq \frac{1}{2k} \forall i \in [1, k]$, it follows that $\text{rank}(\tilde{g}_{nplora}) \leq 3kr$. $\square$

¹³ $\langle g, a_i b_i^\top \rangle = \text{tr}(g^\top a_i b_i^\top)$.

Justification for the $\frac{1}{2k}$ threshold. DPLoRA decouples the parallel branches into k down-projection and k up-projection matrices, each associated with learnable importance factors $\{s_i\}_{i=1}^k$ and $\{s_j\}_{j=1}^k$. To quantify their relative contributions to the gradient subspace, we impose a normalized budget constraint:

$$\sum_{i=1}^k s_i + \sum_{j=1}^k s_j = 1. \quad (\text{A.13})$$

Under a uniform prior, the expected contribution of each matrix is given by

$$\mu = \frac{1}{2k}. \quad (\text{A.14})$$

Accordingly, we employ indicator functions $\mathbf{1}_{\{s_i \geq \mu\}}$ and $\mathbf{1}_{\{s_j \geq \mu\}}$ as an objective pruning criterion. Rather than relying on empirical tuning, this formulation naturally prunes projection matrices whose learned contribution falls below the uniform expectation, effectively filtering out less informative subspace directions.

A.4. Analysis of DPLoRA's equivalent gradient subspace

Dynamic parallel LoRA. To eliminate extra overhead, we sum the down matrices of each branch into an intermediate matrix, then combined the intermediate matrix with the up-projection matrices of the respective branches to construct a single unified LoRA branch, it formulate as:

$$\begin{aligned} \theta^{m \times n} &= \theta_0^{m \times n} + \sum_{j=1}^k \mathbf{1}_{\{s_j \geq \frac{1}{2k}\}} s_j \left(\sum_{i=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} s_i a_i \right) b_j^\top \\ &= \theta_0^{m \times n} + \left(\sum_{i=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} s_i a_i \right) \cdot \left(\sum_{j=1}^k \mathbf{1}_{\{s_j \geq \frac{1}{2k}\}} s_j b_j \right)^\top \\ &= \theta_0^{m \times n} + A_{\text{sum}} \cdot B_{\text{sum}}^\top \end{aligned} \quad (\text{A.15})$$

with:

$$A_{\text{sum}} = \sum_{i=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} s_i a_i, \quad B_{\text{sum}} = \sum_{j=1}^k \mathbf{1}_{\{s_j \geq \frac{1}{2k}\}} s_j b_j$$

DPLoRA update the a_i , b_i , s_i and s_j with respect to

$$\begin{aligned} da_i &= -\frac{\partial \mathcal{L}}{\partial a_i} = -\frac{\partial \mathcal{L}}{\partial \theta} \frac{\partial \theta}{\partial a_i} = -\mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} s_i (g B_{\text{sum}}) = -\mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} g_{a_i}; \\ db_j^\top &= -\frac{\partial \mathcal{L}}{\partial b_j^\top} = -\frac{\partial \mathcal{L}}{\partial \theta} \frac{\partial \theta}{\partial b_j^\top} = -\mathbf{1}_{\{s_j \geq \frac{1}{2k}\}} s_j (A_{\text{sum}}^\top g) = -\mathbf{1}_{\{s_j \geq \frac{1}{2k}\}} g_{b_j}; \\ ds_i &= -\frac{\partial \mathcal{L}}{\partial s_i} = -\sum_{p=1}^m \sum_{q=1}^n \frac{\partial \mathcal{L}}{\partial \theta_{i,pq}} \frac{\partial \theta_{i,pq}}{\partial s_i} = -\mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} \langle g, a_i B_{\text{sum}}^\top \rangle = -\mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} g_{s_i}; \\ ds_j &= -\frac{\partial \mathcal{L}}{\partial s_j} = -\sum_{p=1}^m \sum_{q=1}^n \frac{\partial \mathcal{L}}{\partial \theta_{j,pq}} \frac{\partial \theta_{j,pq}}{\partial s_j} = -\mathbf{1}_{\{s_j \geq \frac{1}{2k}\}} \langle g, A_{\text{sum}} b_j^\top \rangle = -\mathbf{1}_{\{s_j \geq \frac{1}{2k}\}} g_{s_j}; \end{aligned} \quad (\text{A.16})$$

and we can calculate the dA_{sum} and $dB_{\text{sum}}^\top$ as:

$$\begin{aligned} dA_{\text{sum}} &= \sum_{i=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} \left(\frac{\partial A_{\text{sum}}}{\partial a_i} da_i + \frac{\partial A_{\text{sum}}}{\partial s_i} ds_i \right) = -\sum_{i=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} (s_i g_{a_i} + a_i g_{s_i}) \\ dB_{\text{sum}}^\top &= \sum_{j=1}^k \mathbf{1}_{\{s_j \geq \frac{1}{2k}\}} \left(\frac{\partial B_{\text{sum}}^\top}{\partial b_j} db_j + \frac{\partial B_{\text{sum}}^\top}{\partial s_j} ds_j \right) = -\sum_{j=1}^k \mathbf{1}_{\{s_j \geq \frac{1}{2k}\}} (s_j g_{b_j} + b_j^\top g_{s_j}) \end{aligned} \quad (\text{A.17})$$

Therefore, DPLoRA is equivalent to update the $d\theta$ as:

$$\begin{aligned} d\theta &= \frac{\partial \theta}{\partial A_{\text{sum}}} dA_{\text{sum}} + \frac{\partial \theta}{\partial B_{\text{sum}}^\top} dB_{\text{sum}}^\top \\ &= -B_{\text{sum}}^\top \sum_{i=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} (s_i g_{a_i} + a_i g_{s_i}) - A_{\text{sum}} \sum_{j=1}^k \mathbf{1}_{\{s_j \geq \frac{1}{2k}\}} (s_j g_{b_j} + b_j^\top g_{s_j}) \\ &= -\sum_{i=1}^k \sum_{j=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} \mathbf{1}_{\{s_j \geq \frac{1}{2k}\}} \left(s_j (s_i g_{a_i} + a_i g_{s_i}) b_j^\top + s_i a_i (s_j g_{b_j} + b_j^\top g_{s_j}) \right) = -\tilde{g}_{dplora} \end{aligned}$$

Table A.6
Distillation results on LAION-220k-subset.

LAION-Test-Subset		Realistic Vision	SD15	+LoRA32	+LoRA64	+LoRA128	+DPLoRA	BK-SDM-Base	BK-SDM-Small	BK-SDM-Tiny
FID	0	15.40	11.43	10.94	9.49	8.79	10.14	12.44	13.62	
CLIP	0.279	0.268	0.269	0.270	0.271	0.274	0.270	0.262	0.246	

(A.18)

Proposition 3. For DPLoRA with branch-rank r and k branches, the equivalent low-rank gradient has rank at most

$$\text{rank}(\tilde{g}_{\text{dplora}}) \leq \sum_{i=1}^k \sum_{j=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} \mathbf{1}_{\{s_j \geq \frac{1}{2k}\}} \times 4r \quad (\text{A.19})$$

Proof. We use subadditivity and composition properties of matrix rank:

$$\begin{aligned} \text{rank}(\tilde{g}_{\text{dplora}}) &\leq - \sum_{i=1}^k \sum_{j=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} \mathbf{1}_{\{s_j \geq \frac{1}{2k}\}} \left(\text{rank}(s_j s_i g_{a_i} b_j^\top) + \text{rank}(s_j a_i g_{s_i} b_j^\top) \right. \\ &\quad \left. + \text{rank}(s_i a_i s_j g_{b_j}) + \text{rank}(s_i a_i b_j^\top g_{s_j}) \right) \\ &\leq \sum_{i=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} \times (r + r + r + r) = \sum_{i=1}^k \mathbf{1}_{\{s_i \geq \frac{1}{2k}\}} \times (4r) \end{aligned}$$

At initialization, since $\{s_i, s_j\} \geq \frac{1}{2k} \forall i, j \in [1, k]$, it follows that $\text{rank}(\tilde{g}_{\text{dplora}}) \leq 4k^2 r$. $\square$

A.5. More comparison results of DPLoRA

A.5.1. Quantitative comparison

We further construct a LAION test subset outside the LAION training set. As reported in Table A.6, DPLoRA (rank 32, branch 8) achieves FID 8.79 and CLIP 0.274, closely approaching the target Realistic Vision (CLIP 0.279) and outperforming all LoRA and BK-SDM baselines.

A.6. Use of large language models

Large language models (LLMs) were used only for limited writing assistance, including grammatical correction and minor improvements to clarity and phrasing. They played no role in the research design, problem formulation, method development, experiments, analysis, or scientific contributions, and were restricted to surface-level editorial refinement.

Data availability

Data will be made available on request.

References

- [1] J. Ho, A. Jain, P. Abbeel, Denoising diffusion probabilistic models, *Adv. Neural Inf. Process. Syst.* 33 (2020) 6840–6851.
- [2] C. Saharia, W. Chan, S. Saxena, L. Li, J. Whang, E.L. Denton, K. Ghasemipour, R. Gontijo Lopes, B. Karagol Ayan, T. Salimans, et al., Photorealistic text-to-image diffusion models with deep language understanding, *Adv. Neural Inf. Process. Syst.* 35 (2022) 36479–36494.
- [3] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, B. Ommer, High-resolution image synthesis with latent diffusion models, in: *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 10684–10695.
- [4] L. Khachatryan, A. Movsisyan, V. Tadevosyan, R. Henschel, Z. Wang, S. Navasardyan, H. Shi, Text2Video-Zero: Text-to-image diffusion models are zero-shot video generators, in: *IEEE/CVF International Conference on Computer Vision*, 2023, pp. 15954–15964.
- [5] C.-H. Lin, J. Gao, L. Tang, T. Takikawa, X. Zeng, X. Huang, K. Kreis, S. Fidler, M.-Y. Liu, T.-Y. Lin, Magic3D: High-resolution text-to-3d content creation, in: *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 300–309.
- [6] X. Li, Y. Liu, L. Lian, H. Yang, Z. Dong, D. Kang, S. Zhang, K. Keutzer, Q-diffusion: Quantizing diffusion models, in: *IEEE/CVF International Conference on Computer Vision*, 2023, pp. 17535–17545.
- [7] G. Fang, X. Ma, X. Wang, Structural pruning for diffusion models, in: *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2024, pp. 16716–16728.
- [8] T. Salimans, J. Ho, Progressive distillation for fast sampling of diffusion models, in: *International Conference on Learning Representations*, 2022.
- [9] E.J. Hu, yelong shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, LoRA: Low-rank adaptation of large language models, in: *International Conference on Learning Representations*, 2022.
- [10] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, Y. Bengio, Generative adversarial networks, *Commun. ACM* 63 (11) (2020) 139–144.
- [11] T. Karras, S. Laine, T. Aila, A style-based generator architecture for generative adversarial networks, in: *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 4401–4410.
- [12] N. Ruiz, Y. Li, V. Jampani, Y. Pritch, M. Rubinstein, K. Aberman, Dreambooth: Fine tuning text-to-image diffusion models for subject-driven generation, in: *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 22500–22510.
- [13] B.-K. Kim, H.-K. Song, T. Castells, S. Choi, BK-SDM: A lightweight, fast, and cheap version of stable diffusion, in: *European Conference on Computer Vision*, ECCV, 2024, pp. 381–399.
- [14] Y. Zhao, Y. Xu, Z. Xiao, H. Jia, T. Hou, Mobilediffusion: Instant text-to-image generation on mobile devices, in: *European Conference on Computer Vision*, Springer, 2024, pp. 225–242.
- [15] L. Wang, Y. Cheng, S. Li, G. Wu, Y. Wang, J. Yang, WaDi: Weight direction-aware distillation for one-step image synthesis, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 5574–5584.
- [16] H. Han, S. Yeo, J. Yoo, LiFT and PLACE: A simple, stable, and effective knowledge distillation framework for lightweight diffusion models, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 5564–5573.
- [17] L. Zhang, Z. Li, L. Cheng, Q. Zhang, Z. Liu, X. Zhang, C. Xiao, DLIENet: A lightweight low-light image enhancement network via knowledge distillation, *Pattern Recognit.* 169 (2026) 111777, <http://dx.doi.org/10.1016/j.patcog.2025.111777>, URL <https://www.sciencedirect.com/science/article/pii/S0031320325004376>.
- [18] Z. Li, X. Li, L. Yang, R. Song, J. Yang, Z. Pan, Dual teachers for self-knowledge distillation, *Pattern Recognit.* 151 (2024) 110422, <http://dx.doi.org/10.1016/j.patcog.2024.110422>, URL <https://www.sciencedirect.com/science/article/pii/S0031320324001730>.
- [19] H. Luo, S. Wan, X. Cai, C. Wang, H. Chen, Dual-hierarchical knowledge distillation for video captioning, *Pattern Recognit.* 171 (2026) 112192, <http://dx.doi.org/10.1016/j.patcog.2025.112192>, URL <https://www.sciencedirect.com/science/article/pii/S0031320325008532>.
- [20] Z. Wang, J. Liang, R. He, Z. Wang, T. Tan, LoRA-pro: Are low-rank adapters properly optimized? in: *The Thirteenth International Conference on Learning Representations*, 2025.
- [21] S. Ling, L. Zhang, J. Zhao, L. Pan, H. Li, LoRA-based continual learning with constraints on critical parameter changes, *Pattern Recognit.* 171 (2026) 112086, <http://dx.doi.org/10.1016/j.patcog.2025.112086>, URL <https://www.sciencedirect.com/science/article/pii/S0031320325007460>.
- [22] K. Lv, W. Fan, H. Cao, K. Tu, Y. Xu, Z. Zhang, Y. Li, X. Ding, Y. Wang, Hyper adversarial tuning for boosting adversarial robustness of pretrained large vision transformers, *Pattern Recognit.* 171 (2026) 112158, <http://dx.doi.org/10.1016/j.patcog.2025.112158>, URL <https://www.sciencedirect.com/science/article/pii/S0031320325008180>.
- [23] M. Li, Y. Chen, Z. Rao, C. Jiang, K. Wei, J. Guo, SG-LoRA: Semantic-guided LoRA parameters generation, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 22206–22216.
- [24] X. Zhu, D. Liang, X. Jiang, Y. Guan, Y. Liu, Y. Zhu, X. Bai, Layerlink: Bridging remote sensing object detection and large vision models with efficient fine-tuning, *Pattern Recognit.* 165 (2025) 111583, <http://dx.doi.org/10.1016/j.patcog.2025.111583>, URL <https://www.sciencedirect.com/science/article/pii/S0031320325002432>.
- [25] J. Xing, J. Zhao, C. Xu, M. Wang, G. Dai, Y. Liu, J. Wang, X. Li, MA-FSAR: Multimodal adaptation of CLIP for few-shot action recognition, *Pattern Recognit.* 169 (2026) 111902, <http://dx.doi.org/10.1016/j.patcog.2025.111902>, URL <https://www.sciencedirect.com/science/article/pii/S003132032500562X>.

- [26] A. Aghajanyan, L. Zettlemoyer, S. Gupta, Intrinsic dimensionality explains the effectiveness of language model fine-tuning, in: 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, Association for Computational Linguistics, Online, 2021, pp. 7319–7328.
- [27] Q. Zhang, M. Chen, A. Bukharin, P. He, Y. Cheng, W. Chen, T. Zhao, Adaptive budget allocation for parameter-efficient fine-tuning, in: The Eleventh International Conference on Learning Representations, 2023.
- [28] X. Ding, X. Zhang, N. Ma, J. Han, G. Ding, J. Sun, Repvgg: Making vgg-style convnets great again, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2021, pp. 13733–13742.
- [29] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, C.L. Zitnick, Microsoft COCO: Common objects in context, in: European Conference on Computer Vision, Springer, 2014, pp. 740–755.
- [30] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, S. Hochreiter, Gans trained by a two time-scale update rule converge to a local nash equilibrium, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [31] J. Hessel, A. Holtzman, M. Forbes, R.L. Bras, Y. Choi, CLIPScore: A reference-free evaluation metric for image captioning, in: The 2021 Conference on Empirical Methods in Natural Language Processing, 2021.
- [32] M. Chen, J. Tworek, H. Jun, Q. Yuan, H.P.D.O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al., Evaluating large language models trained on code, 2021, arXiv preprint [arXiv:2107.03374](https://arxiv.org/abs/2107.03374).
- [33] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al., Training verifiers to solve math word problems, 2021, arXiv preprint [arXiv:2110.14168](https://arxiv.org/abs/2110.14168).
- [34] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, et al., Judging llm-as-a-judge with mt-bench and chatbot arena, *Adv. Neural Inf. Process. Syst.* 36 (2023) 46595–46623.
- [35] D. Kalajdziewski, A rank stabilization scaling factor for fine-tuning with lora, 2023, arXiv preprint [arXiv:2312.03732](https://arxiv.org/abs/2312.03732).
- [36] F. Meng, Z. Wang, M. Zhang, Pissa: Principal singular values and singular vectors adaptation of large language models, *Adv. Neural Inf. Process. Syst.* 37 (2024) 121038–121072.
- [37] S. Hayou, N. Ghosh, B. Yu, LoRA+ efficient low rank adaptation of large models, in: Proceedings of the 41st International Conference on Machine Learning, 2024, pp. 17783–17806.
- [38] S.-Y. Liu, C.-Y. Wang, H. Yin, P. Molchanov, Y.-C.F. Wang, K.-T. Cheng, M.-H. Chen, DoRA: Weight-decomposed low-rank adaptation, in: International Conference on Machine Learning, PMLR, 2024, pp. 32100–32121.
- [39] S. Wang, L. Yu, J. Li, Lora-ga: Low-rank adaptation with gradient approximation, *Adv. Neural Inf. Process. Syst.* 37 (2024) 54905–54931.